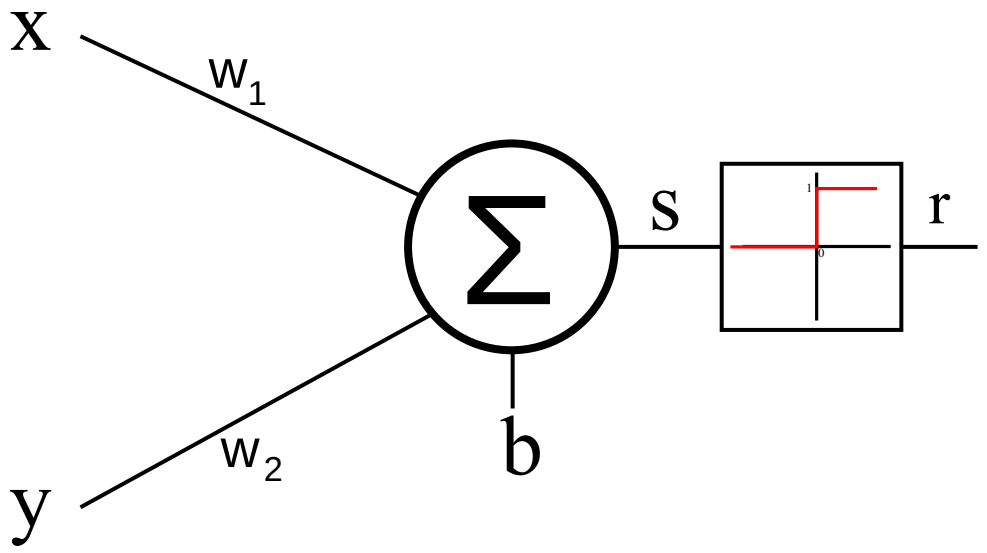
Artificial Intelligence for robotics

Part 5 : Multi-layer neural networks

Simon Gay

Artificial Intelligence for Robotics

- **Limits of single-layer networks**
  - a short case study: logic gates
    - We want to create logic gates AND, OR and XOR with formal neurons
    - A neuron per gate
      - Two inputs x and y and a bias b
      - Threshold activation function: 1 if sum >0, 0 else



Artificial Intelligence for Robotics

- **OR gate**

Table de vérité de OU		
0	0	0
0	1	1
1	0	1
1	1	1

- **A solution (among other): w1=3, w2=3, b=-1**
  - **0 . 3 + 0 . 3 - 1 = -1 → 0**
  - **0 . 3 + 1 . 3 - 1 = 2 → 1**
  - **1 . 3 + 0 . 3 - 1 = 2 → 1**
  - **1 . 3 + 1 . 3 - 1 = 5 → 1**

Artificial Intelligence for Robotics

- **AND gate**

Table de vérité de ET		
0	0	0
0	1	0
1	0	0
1	1	1

- **A solution (among other): w1=2, w2=2, b=-3**
  - **0 . 2 + 0 . 2 - 3 = -3 → 0**
  - **0 . 2 + 1 . 2 - 3 = -1 → 0**
  - **1 . 2 + 0 . 2 - 3 = -1 → 0**
  - **1 . 2 + 1 . 2 - 3 = 1 → 1**

Artificial Intelligence for Robotics

- **XOR gate**

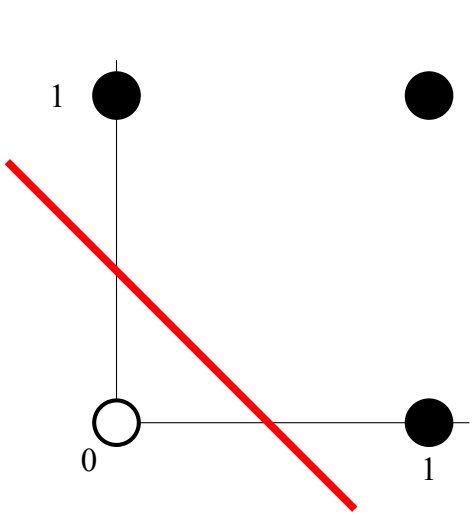
Table de vérité de XOR		
0	0	0
0	1	1
1	0	1
1	1	0

- **Solution:**

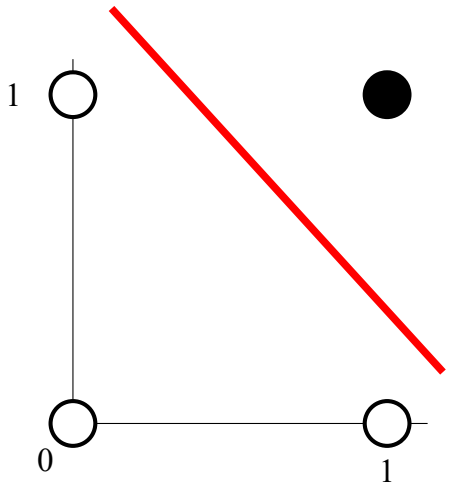
?

Artificial Intelligence for Robotics

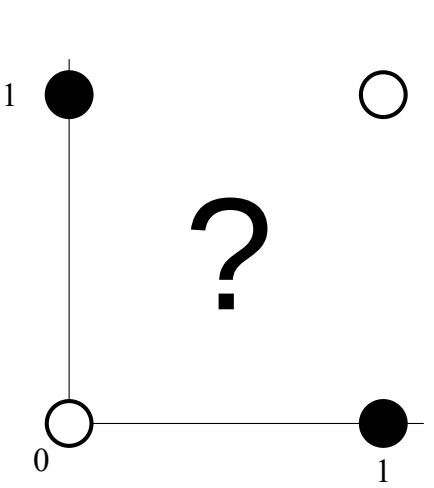
- **Explanation**



OR



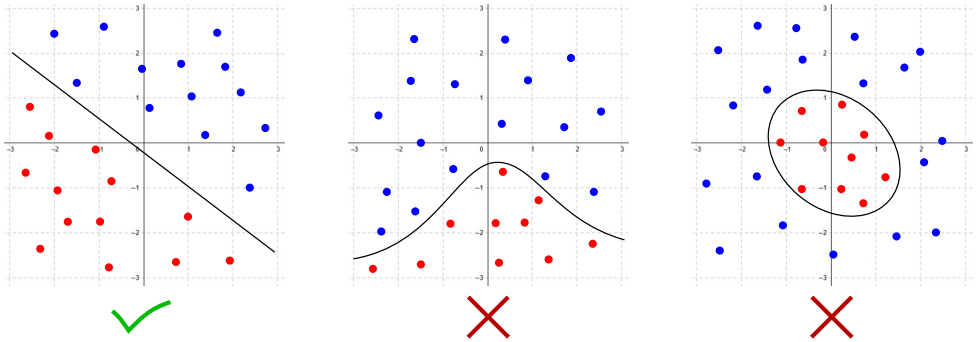
AND



XOR

Artificial Intelligence for Robotics

- **Single-Layer networks are limited to linear problems**



- In 1969, the book *Perceptrons* (from Marvin Minsky and Seymour Papert) implyed that perceptrons will never overcome this limit

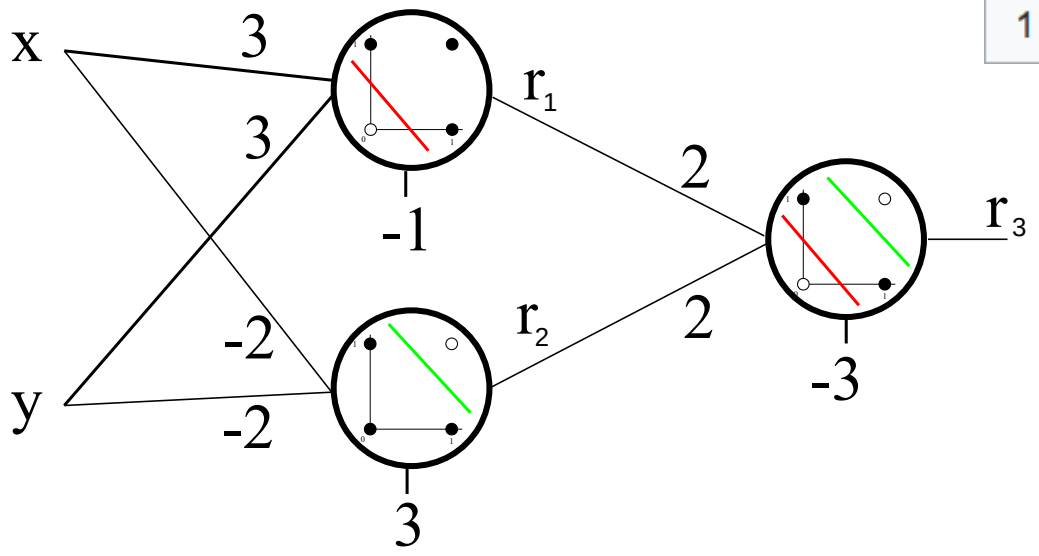
  → decline in research in this area (area of symbolic AI)

Artificial Intelligence for Robotics

- **A solution for XOR gate**

  → **with multiple layers**

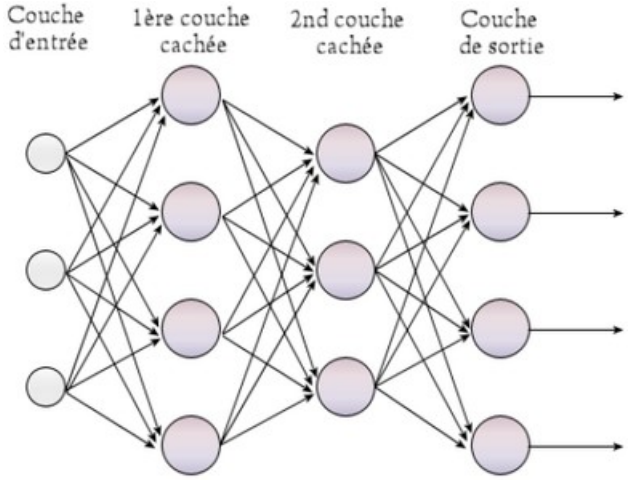
Table de vérité de XOR		
0	0	0
0	1	1
1	0	1
1	1	0



- Neurons of next layer can combine “separators” of neurons of previous layers, and solve non-linear problems

Artificial Intelligence for Robotics

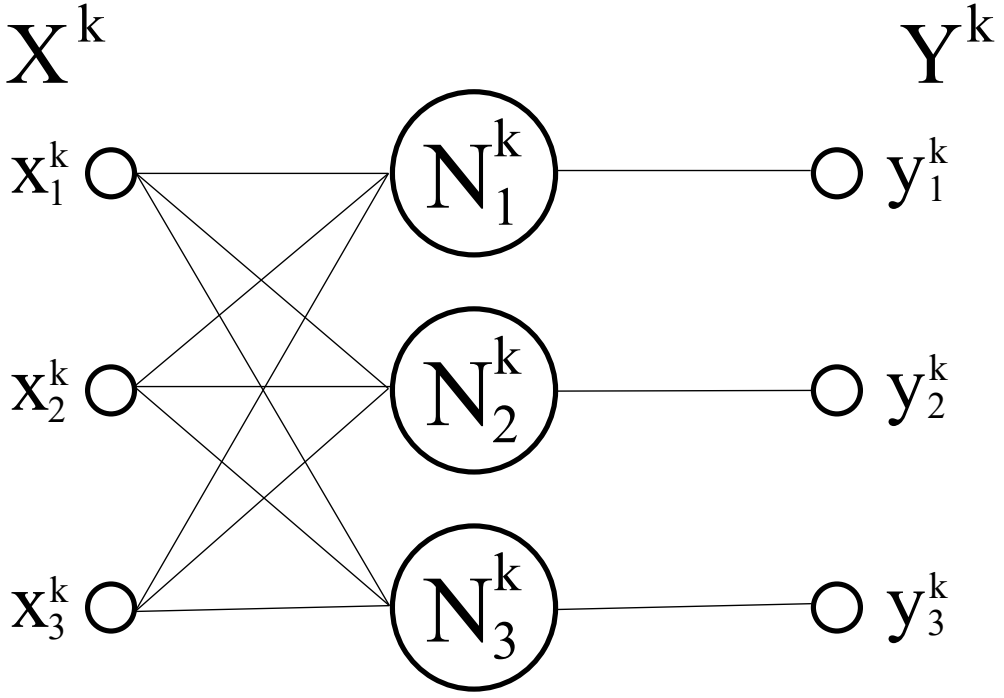
- **Multi-Layer networks**
  - First multi-layer network created in 1986 by David Rumelhart (nearly 30 years after the perceptron !)
  - Composed of:
    - An input layer (input vector)
    - One or more hidden layer
    - An output layer



  - How to calculate the network result ?
  - How updating weights ?

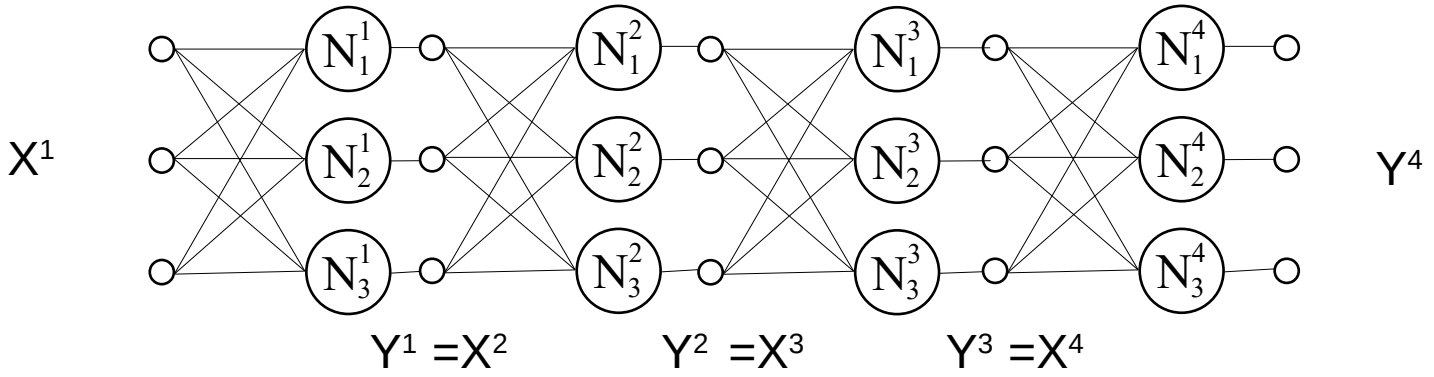
Artificial Intelligence for Robotics

- **How to calculate the network result ? Let's normalize notations**
  - We note N_i^k the ith neuron in layer k
  - A neuron uses as input a vector $X^k = \{x_1^k, x_2^k, \dots, x_n^k\}$ and has an output y_i^k
    - Each layer has as input a vector X^k and an output vector Y^k



Artificial Intelligence for Robotics

- **How to calculate the network result ?**
  - The network uses vector X^1 as input
  - First hidden layer compute output vector y^1
  - Next layer use as input the previous layer's output vector
    - thus, $X^k = Y^{k-1}$ (and $x_i^k = y_i^{k-1}\ \forall i$)
  - The process is repeated until output layer



→ no changes compared with single layer networks

Artificial Intelligence for Robotics

- **How updating weights ?**

  → It is far more complex !

  - Each change in a weight will affect all neurons in all next layers

  - We need to define the impact of a weight change on network outputs

$$\frac{\partial y_i^L}{\partial w_{ij}^k}$$

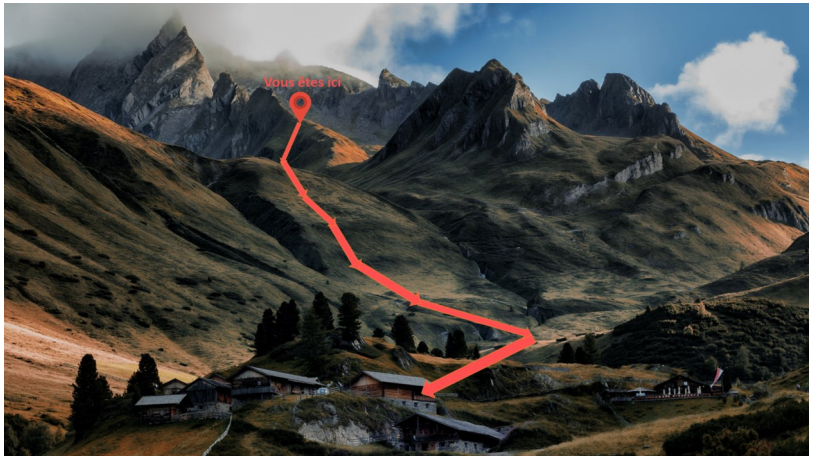
  - Algorithm of *gradient descent*

Artificial Intelligence for Robotics

- **Gradient descent**

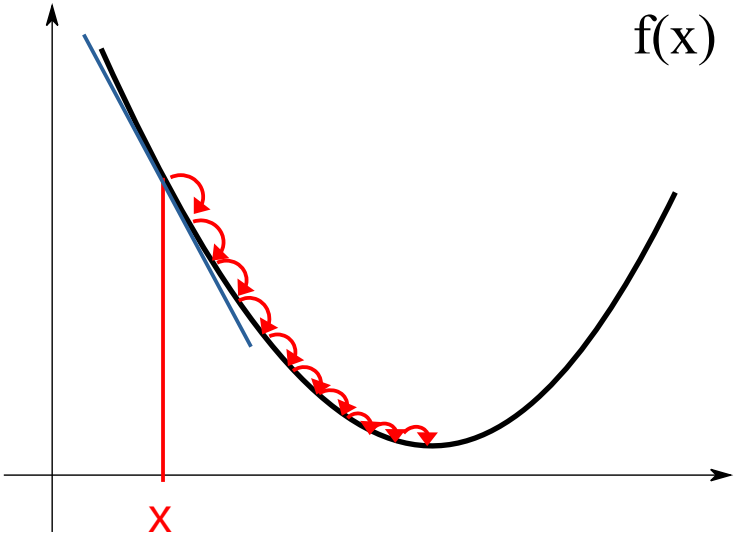
  - Illustration: mountain hike
    - Suddenly: a blizzard, visibility reduced to a few meters
    - The mountain refuge is at the lowest point in the valley
    - You are still in the valley

  - How do you get back to the refuge?
    - Get the direction of the steepest descent
    - Move forward a hundred meters
    - Repeat until reaching the bottom of the valley



Artificial Intelligence for Robotics

- **Gradient descent**



- A function f to minimize (min($f_{(x)}$))
- Start with a random x^0 value
- Calculate the gradient of f in x ($f'_{(x)}$)
- Apply an update on x defined by:

$$x^{t+1} = x^t - \alpha . \frac{df(x)}{dx}$$

- Repeat until convergence

Artificial Intelligence for Robotics

- **Gradient descent applied on neural networks**
  - We define an error function:

$$E(y) = \frac{1}{2} . (r - y)^2$$

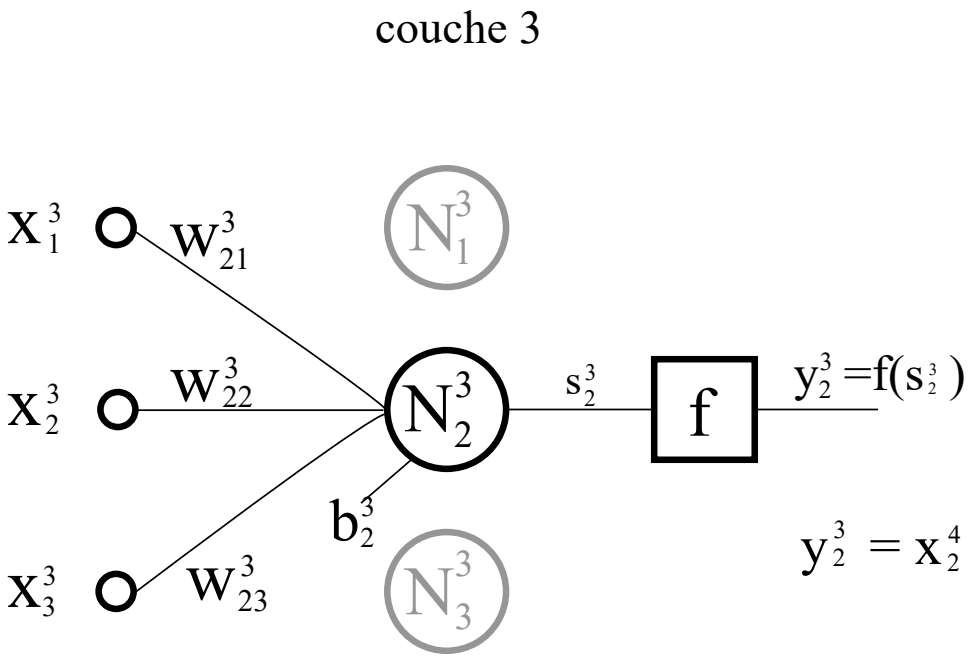
  - For each weight w_{ij}^l , we determine its impact on E
    - Gradient descent on w_{ij}^l

$$w_{ij}^l \Leftarrow w_{ij}^l - \alpha . \frac{\partial E(y)}{\partial w_{ij}^l}$$

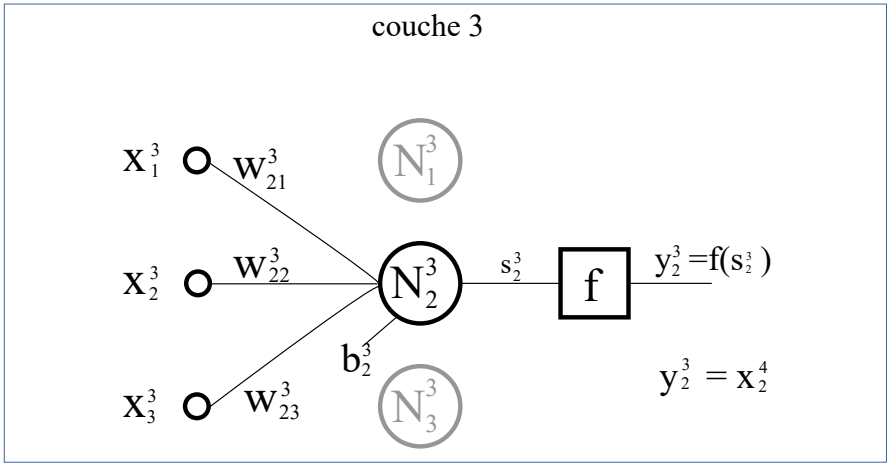
    - We thus need to calculate $\frac{\partial E(y)}{\partial w_{ij}^l}$

Artificial Intelligence for Robotics

- **Gradient calculation: clarification on the notations**
  - We note l a layer, i the index of a neuron, L the output layer.
  - A weight w_{ij}^{l} connects the neuron N_i^l and the neuron N_j^{l-1} ($N_j^{l-1} \rightarrow N_i^l$)
  - We note $s_i^l = s(W_i^l, X^l, b_i^l) = \Sigma\, w_{ij}^l \,.\, x_i^l + b_i^l$ the weighted sum of neuron N_i^l of layer l.
  - We note $y_i^l = f(s_i^l)$ the output of neuron N_i^l after activation function f



Artificial Intelligence for Robotics



- **Gradient calculation:**
  case of last layer L
  - We need to find

$$\frac{\partial E\left(y_i^L\right)}{\partial w_{ij}^L}$$

  - We decompose through intermediate variables
    - Derivative Rule for Composite Functions

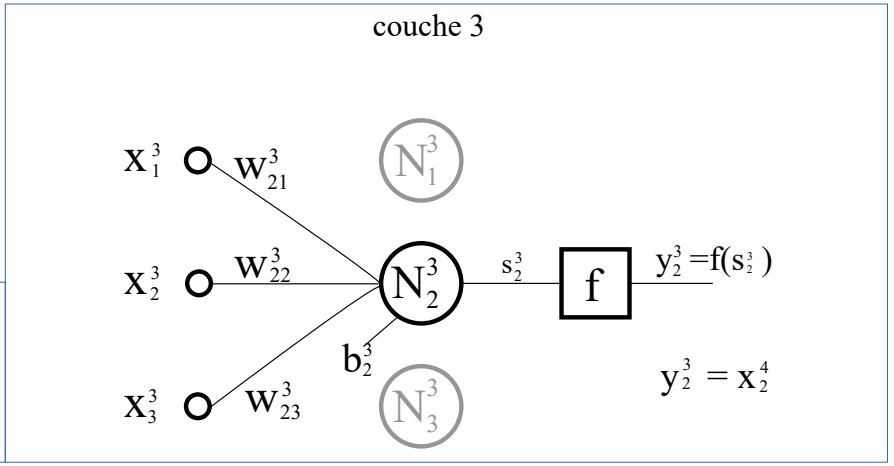
$$\frac{\partial E\left(y_i^L\right)}{\partial w_{ij}^L} = \frac{\partial E\left(y_i^L\right)}{\partial y_i^L} \cdot \frac{\partial y_i^L}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^L}$$

    - Three derivatives to find

Artificial Intelligence for Robotics

- **Gradient calculation:**
  case of last layer L

$$\frac{\partial E(y_i^L)}{\partial w_{ij}^L} = \frac{\partial E(y_i^L)}{\partial y_i^L} \cdot \frac{\partial y_i^L}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^L}$$



- First term: $\frac{\partial E(y_i^L)}{\partial y_i^L}$

- Error function: $E(y) = \frac{1}{2}.(r - y)^2$

Derivative of composite functions:
(f o g)' = (f' o g).g'

$f(x) = x^2$
$g(x) = r - x$

-1

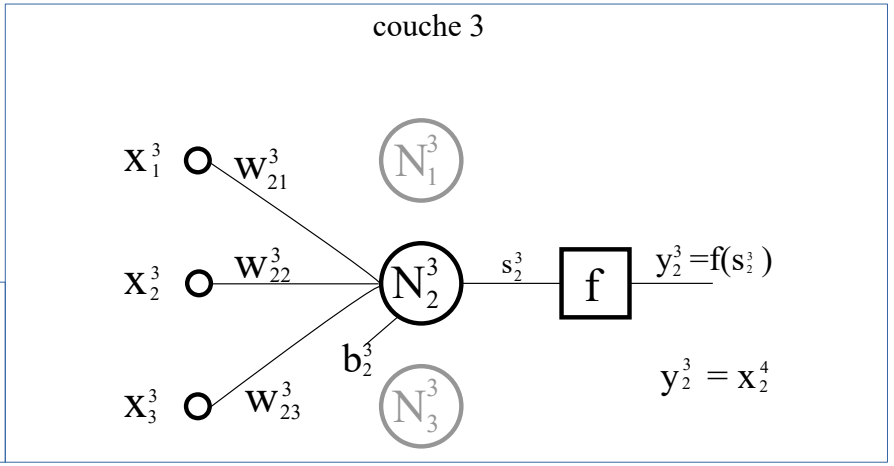
$$\frac{\partial E(y)}{\partial y} = \frac{1}{2}.\frac{\partial (r-y)^2}{\partial y} = \frac{2}{2}.(r-y).\frac{\partial (r-y)}{\partial y}$$

$$\frac{\partial E(y)}{\partial y} = -(r-y)$$

Artificial Intelligence for Robotics

- **Gradient calculation:**
  case of last layer L

$$\frac{\partial E(y_i^L)}{\partial w_{ij}^L} = \frac{\partial E(y_i^L)}{\partial y_i^L} \cdot \frac{\partial y_i^L}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^L}$$



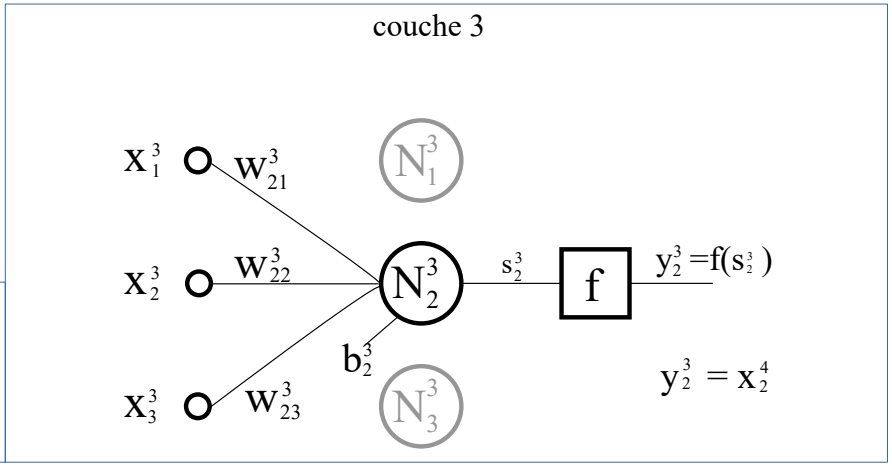
- second term: $\frac{\partial y_i^L}{\partial s_i^L} = \frac{\partial f(s_i^L)}{\partial s_i^L}$

- Simply the derivative of the activation function !

$$\frac{\partial y_i^L}{\partial s_i^L} = f'_{(s_i^L)}$$

Artificial Intelligence for Robotics

- **Gradient calculation:**
  case of last layer L



$$\frac{\partial E(y_i^L)}{\partial w_{ij}^L} = \frac{\partial E(y_i^L)}{\partial y_i^L} \cdot \frac{\partial y_i^L}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^L}$$

- Third term: $\dfrac{\partial s_i^L}{\partial w_{ij}^L}$

'constants'

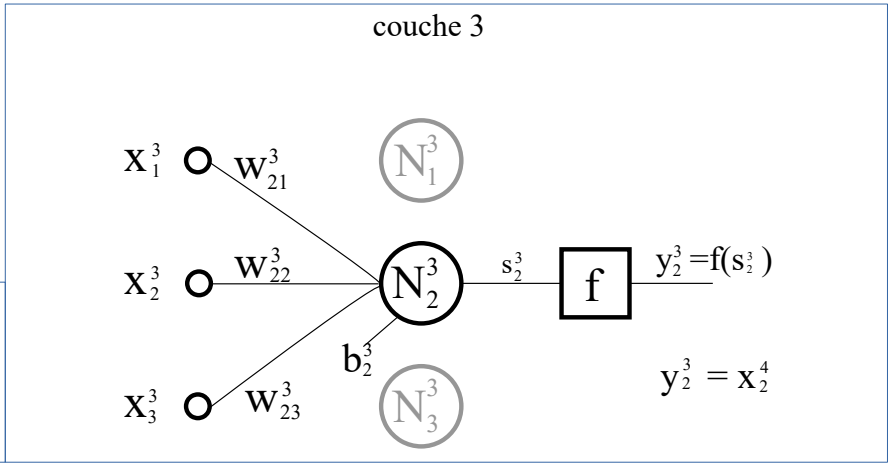
$$\frac{\partial s_i^L}{\partial w_{ij}^L} = \frac{\partial \left(w_{i1}^L . x_1^L + w_{i2}^L . x_2^L + \ldots + w_{ij}^L . x_i^L + \ldots + b_i^L \right)}{\partial w_{ij}^L} = \frac{\partial \left(w_{ij}^L . x_j^L \right)}{\partial w_{ij}^L}$$

$$\frac{\partial s_i^L}{\partial w_{ij}^L} = x_j^L$$

Artificial Intelligence for Robotics

- **Gradient calculation:**
  case of last layer L

$$\frac{\partial E(y_i^L)}{\partial w_{ij}^L} = \frac{\partial E(y_i^L)}{\partial y_i^L} \cdot \frac{\partial y_i^L}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^L}$$



- Let's recap:

$$\frac{\partial E(y_i^L)}{\partial w_{ij}^L} = -(r - y_i^L) \cdot f'_{(s_i^L)} \cdot x_j^L$$

- Gradient descent: $w_{ij}^L \Leftarrow w_{ij}^L + \alpha . x_j^L . (r - y_i^L) . f'_{(s_i^L)}$

- We note $\delta_i^L = (r - y_i^L) \cdot f'_{(s(...))}$

→ $\boxed{w_{ij}^L \Leftarrow w_{ij}^L + \alpha . x_j^L . \delta_i^L}$

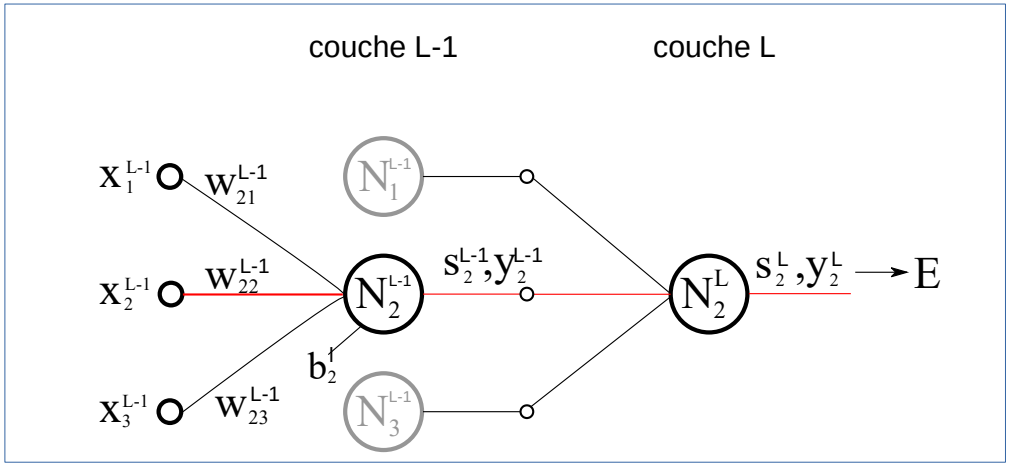
We can notice:

$$\delta_i^L = -\frac{\partial E(y_i^L)}{\partial s_i^L}$$

Artificial Intelligence for Robotics

- **Gradient calculation:**
  case of hidden layer L-1
  - We want to calculate

$$\frac{\partial E\left(y_m^L\right)}{\partial w_{ij}^{L-1}}$$



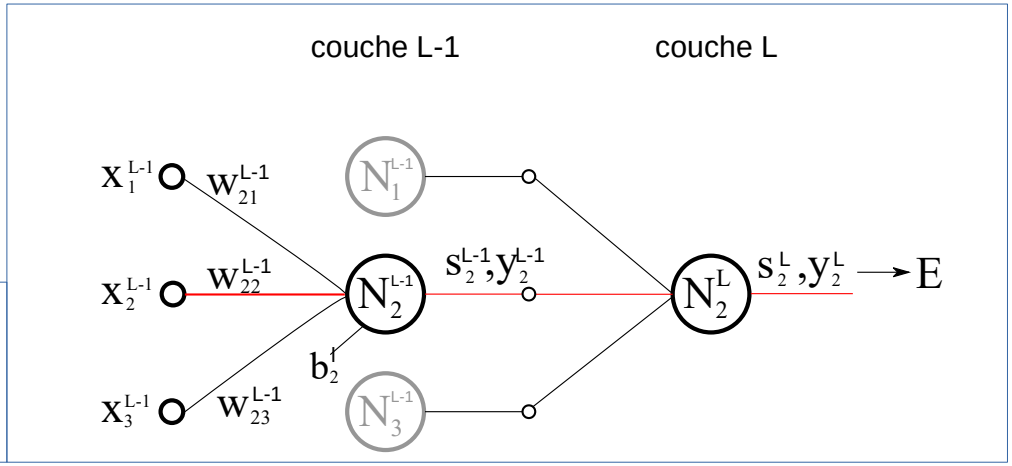
  - We decompose again:

$$\frac{\partial E\left(y_m^L\right)}{\partial w_{ij}^{L-1}} = \frac{\partial E\left(y_m^L\right)}{\partial y_i^{L-1}} \cdot \frac{\partial y_i^{L-1}}{\partial s_i^{L-1}} \cdot \frac{\partial s_i^{L-1}}{\partial w_{ij}^{L-1}}$$

Artificial Intelligence for Robotics

- **Gradient calculation:**
  case of hidden layer L-1

$$\frac{\partial E(y_m^L)}{\partial w_{ij}^{L-1}} = \frac{\partial E(y_m^L)}{\partial y_i^{L-1}} \cdot \frac{\partial y_i^{L-1}}{\partial s_i^{L-1}} \cdot \frac{\partial s_i^{L-1}}{\partial w_{ij}^{L-1}}$$



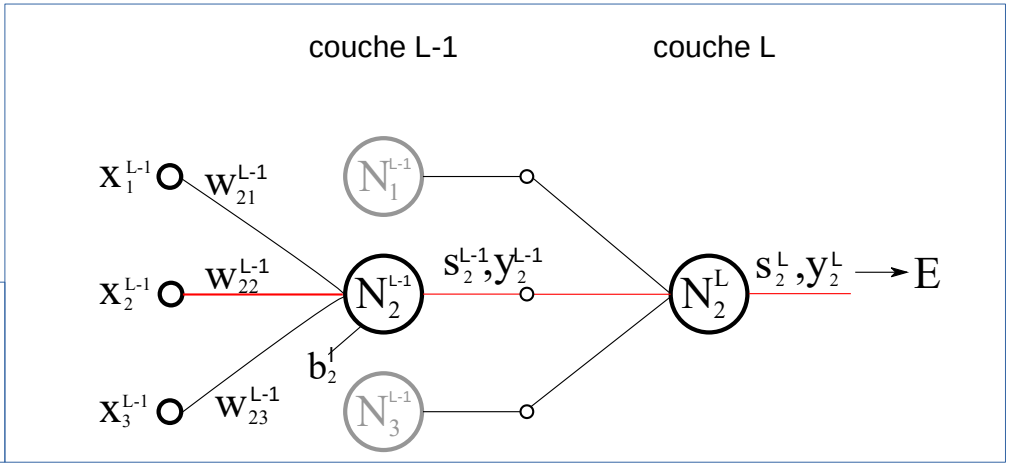
- Second and third terms are the same

$$\frac{\partial y_i^{L-1}}{\partial s_i^{L-1}} = f'_{(s_i^{L-1})}$$

$$\frac{\partial s_i^{L-1}}{\partial w_{ij}^{L-1}} = x_j^{L-1}$$

Artificial Intelligence for Robotics

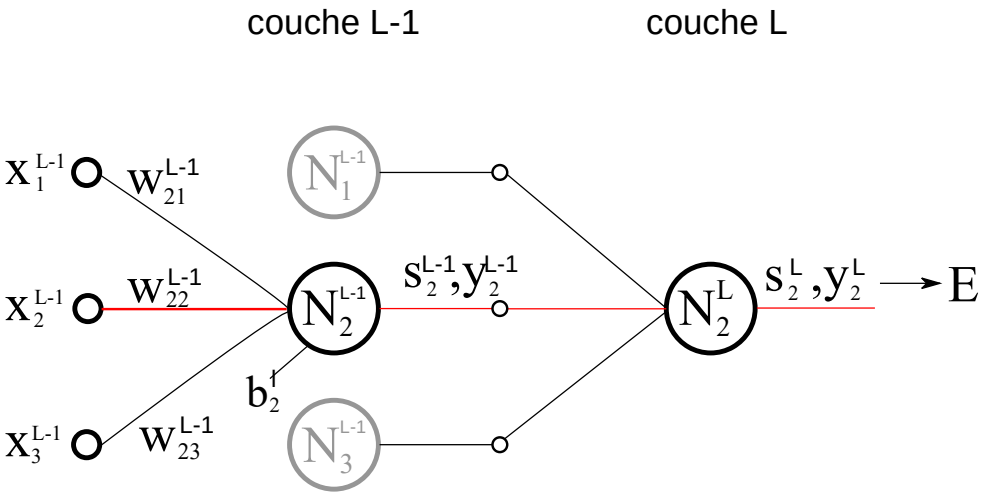
- **Gradient calculation:**
  case of hidden layer L-1



$$\frac{\partial E(y_m^L)}{\partial w_{ij}^{L-1}} = \frac{\partial E(y_m^L)}{\partial y_i^{L-1}} \cdot \frac{\partial y_i^{L-1}}{\partial s_i^{L-1}} \cdot \frac{\partial s_i^{L-1}}{\partial w_{ij}^{L-1}}$$

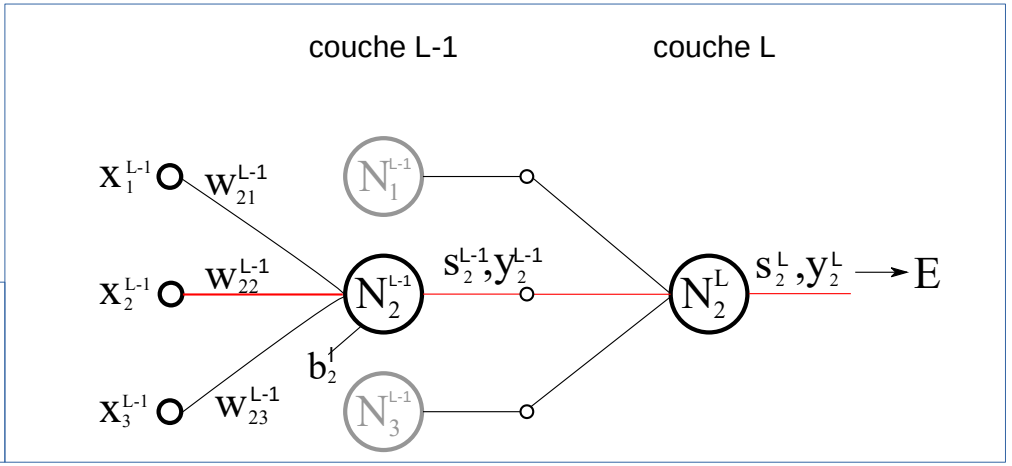
- First term: $\dfrac{\partial E(y_m^L)}{\partial y_i^{L-1}}$

- A little bit more complex: if y_i^{L-1} changes, E is modified through s_i^L (and thus y_j^L) of layer L



Artificial Intelligence for Robotics

- **Gradient calculation:**
  case of hidden layer L-1



$$\frac{\partial E(y_m^L)}{\partial w_{ij}^{L-1}} = \frac{\partial E(y_m^L)}{\partial y_i^{L-1}} \cdot \frac{\partial y_i^{L-1}}{\partial s_i^{L-1}} \cdot \frac{\partial s_i^{L-1}}{\partial w_{ij}^{L-1}}$$

$$\frac{\partial E(y_m^L)}{\partial y_i^{L-1}} = \frac{\partial E(y_m^L)}{\partial s_m^L} \cdot \frac{\partial s_m^L}{\partial y_i^{L-1}}$$

It's $-\delta_m^L$!

$$\frac{\partial s_m^L}{\partial y_i^{L-1}} = \frac{\partial (w_{m1}^L . x_1^L + w_{m2}^L . x_2^L + \ldots + w_{mi}^L . x_i^L + \ldots + b_m^L)}{\partial x_i^L} = w_{mi}^L$$

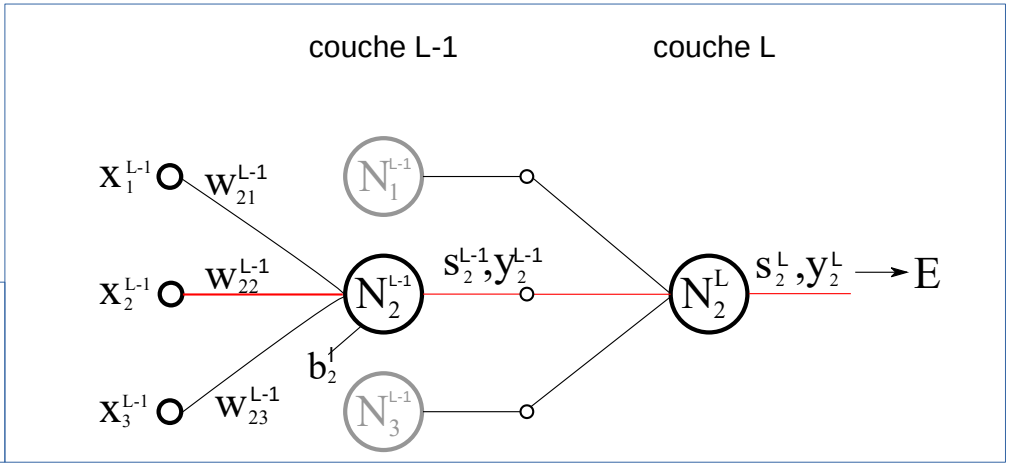
$y_i^{L-1} = x_i^L$

$$\frac{\partial E(y_m^l)}{\partial y_i^{L-1}} = -\delta_m^L \cdot w_{mi}^L$$

Artificial Intelligence for Robotics

- **Gradient calculation:**
  case of hidden layer L-1

$$\frac{\partial E(y_m^L)}{\partial w_{ij}^{L-1}} = \frac{\partial E(y_m^L)}{\partial y_i^{L-1}} \cdot \frac{\partial y_i^{L-1}}{\partial s_i^{L-1}} \cdot \frac{\partial s_i^{L-1}}{\partial w_{ij}^{L-1}}$$



- Let's recap:

$$\frac{\partial E(y_i^L)}{\partial w_{ij}^{L-1}} = -\, x_j^{L-1} \cdot f'_{(s_i^{L-1})} \cdot \delta_m^L \cdot w_{mi}^L$$

- Gradient descent: $w_{ij}^{L-1} \Leftarrow w_{ij}^{L-1} + \alpha . x_j^{L-1} . f'_{(s_i^{L-1})} \cdot \delta_m^L \cdot w_{mi}^L$

- We note: $\delta_i^{L-1} = f'_{(s_i^{L-1})} \cdot \delta_m^L \cdot w_{mi}^L$

We can notice:

$$\delta_i^{L-1} = -\frac{\partial E(y_m^L)}{\partial s_i^{L-1}}$$

We still have $\boxed{w_{ij}^{L-1} \Leftarrow w_{ij}^{L-1} + \alpha . x_j^{L-1} . \delta_i^{L-1}}$

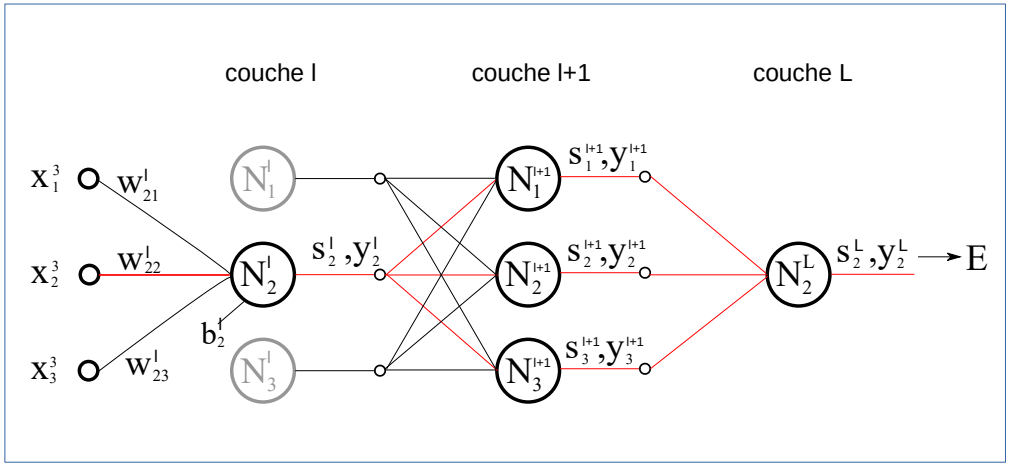
Artificial Intelligence for Robotics

- **Gradient calculation:**

  Generalization for layers l

  - We want to calculate

$$\frac{\partial E\left(y_m^L\right)}{\partial w_{ij}^l}$$



  - We decompose again:

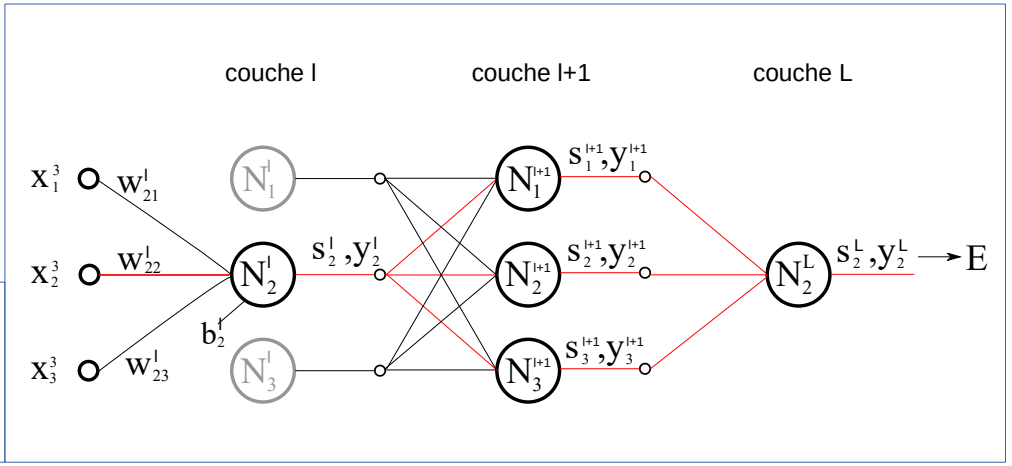
$$\frac{\partial E\left(y_m^L\right)}{\partial w_{ij}^l} = \frac{\partial E\left(y_m^L\right)}{\partial y_i^l} \cdot \frac{\partial y_i^l}{\partial s_i^l} \cdot \frac{\partial s_i^l}{\partial w_{ij}^l}$$

Artificial Intelligence for Robotics

- **Gradient calculation:**

  Generalization for layers l

$$\frac{\partial E(y_m^L)}{\partial w_{ij}^l} = \frac{\partial E(y_m^L)}{\partial y_i^l} \cdot \frac{\partial y_i^l}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^l}$$



  - Still no changes in second and third terms

$$\frac{\partial y_i^l}{\partial s_i^l} = f'_{(s_i^l)}$$

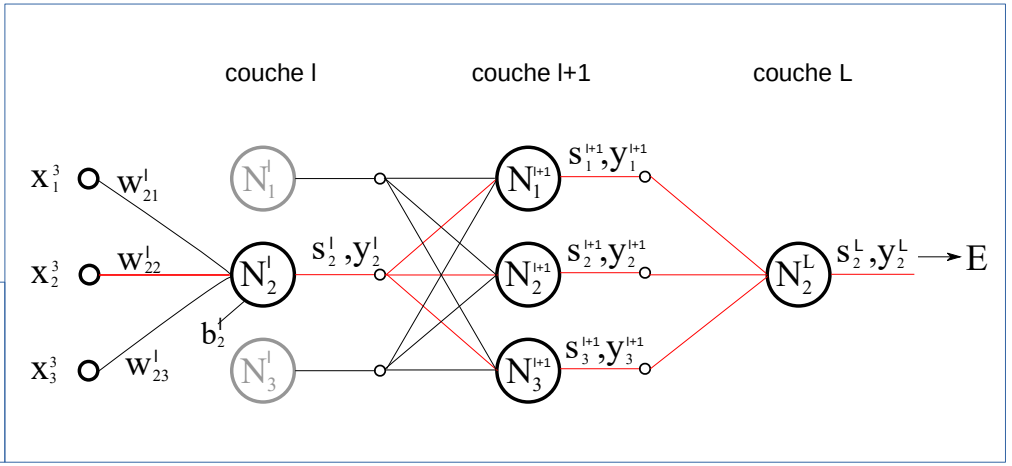
$$\frac{\partial s_i^l}{\partial w_{ij}^l} = x_j^l$$

Artificial Intelligence for Robotics

- **Gradient calculation:**

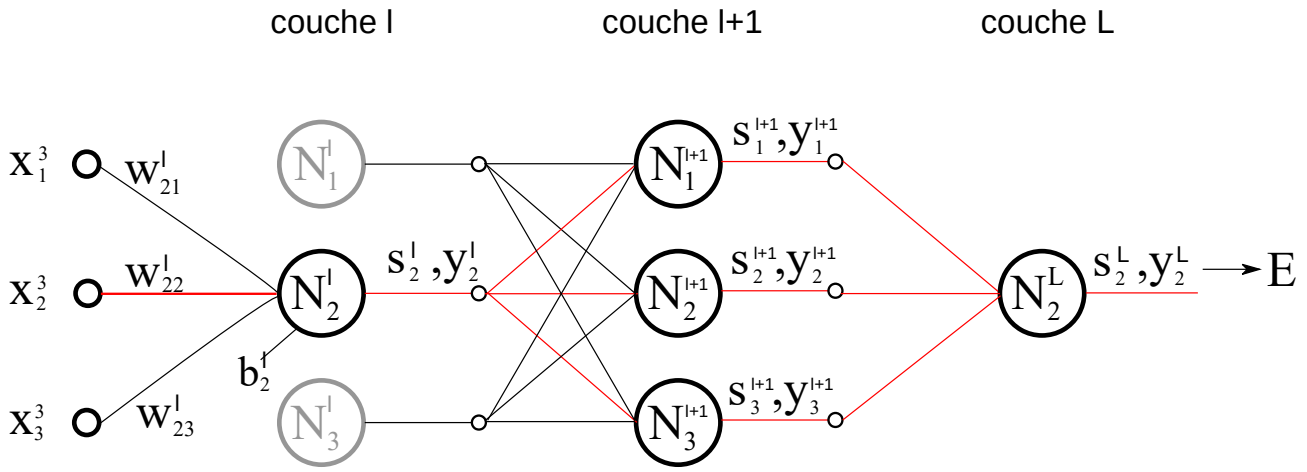
  Generalization for layers l

$$\frac{\partial E(y_m^L)}{\partial w_{ij}^l} = \frac{\partial E(y_m^L)}{\partial y_i^l} \cdot \frac{\partial y_i^l}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^l}$$



  - First term: $\frac{\partial E(y_m^L)}{\partial y_i^l}$

  - This time, if y_i^l changes, E is modified through outputs s_j^{l+1} (and y_j^{l+1}) of next layer l+1

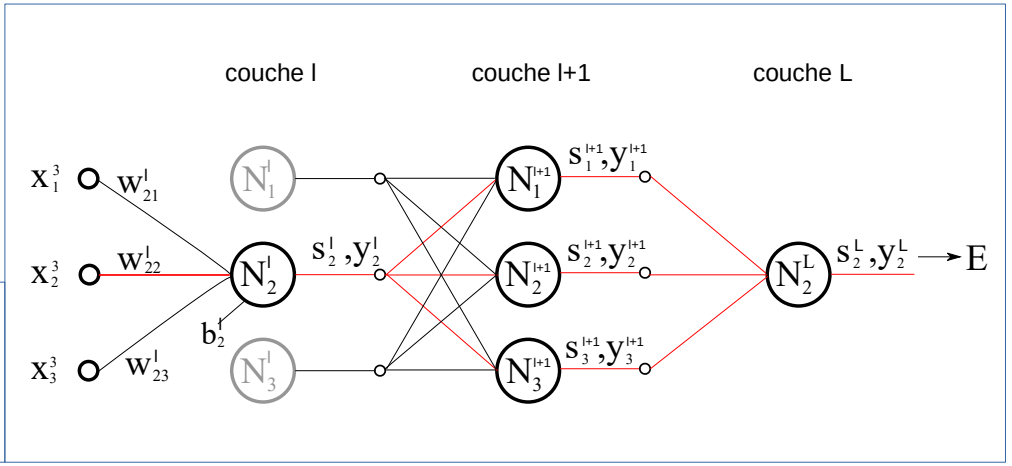


Artificial Intelligence for Robotics

- **Gradient calculation:**

  Generalization for layers l

$$\frac{\partial E(y_m^L)}{\partial w_{ij}^l} = \frac{\partial E(y_m^L)}{\partial y_i^l} \cdot \frac{\partial y_i^l}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^l}$$



$$\frac{\partial E(y_m^L)}{\partial y_i^l} = \frac{\partial E(y_m^L)}{\partial s_1^{l+1}} \cdot \frac{\partial s_1^{l+1}}{\partial y_i^l} + \frac{\partial E(y_m^L)}{\partial s_2^{l+1}} \cdot \frac{\partial s_2^{l+1}}{\partial y_i^l} + \dots + \frac{\partial E(y_m^L)}{\partial s_n^{l+1}} \cdot \frac{\partial s_n^{l+1}}{\partial y_i^l}$$

$$\frac{\partial E(y_m^l)}{\partial y_i^l} = \sum_{k \in [1,n]} \frac{\partial E(y_m^L)}{\partial s_k^{l+1}} \cdot \frac{\partial s_k^{l+1}}{\partial y_i^l}$$

$\frac{\partial s_k^{l+1}}{\partial y_i^l} \rightarrow w_{ki}^{l+1}$

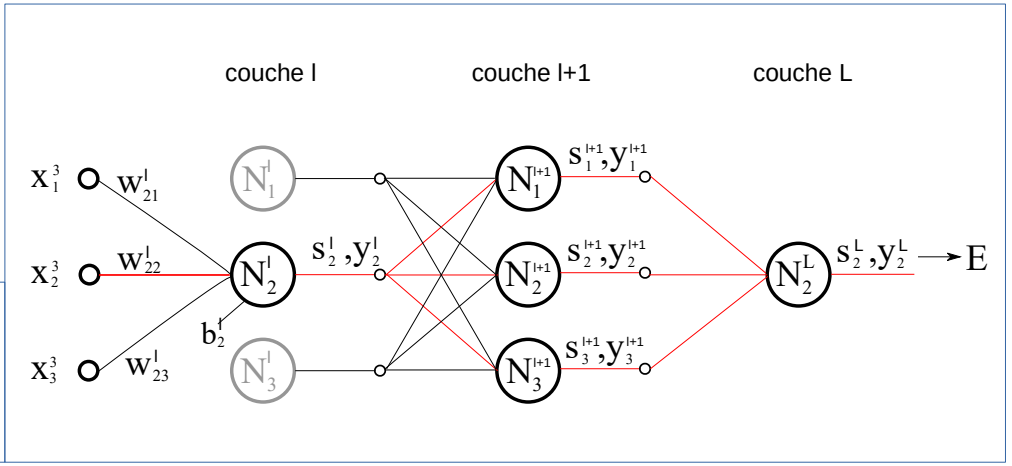
$\frac{\partial E(y_m^L)}{\partial s_k^{l+1}}$ → They are $-\delta_k^{l+1}$!

Artificial Intelligence for Robotics

- **Gradient calculation:**

  Generalization for layers l

$$\frac{\partial E(y_m^L)}{\partial w_{ij}^l} = \frac{\partial E(y_m^L)}{\partial y_i^l} \cdot \frac{\partial y_i^l}{\partial s_i^L} \cdot \frac{\partial s_i^L}{\partial w_{ij}^l}$$



- Let's recap (a last time):

$$\frac{\partial E(y_i^L)}{\partial w_{ij}^l} = -x_j^l \cdot f'_{(s_i^l)} \cdot \sum_{k\in[1,n]} \delta_k^{l+1} \cdot w_{ki}^{l+1}$$

- Gradient descent:

$$w_{ij}^l \Leftarrow w_{ij}^l + \alpha . x_j^l . f'_{(s_i^l)} \cdot \sum_{k\in[1,n]} \delta_k^{l+1} . w_{ki}^{l+1}$$

- We note:

$$\delta_i^l = f'_{(s_i^l)} \cdot \sum_{k\in[1,n]} \delta_k^{l+1} \cdot w_{ki}^{l+1}$$

and again:

$$w_{ij}^l \Leftarrow w_{ij}^l + \alpha . x_j^l . \delta_i^l$$

Artificial Intelligence for Robotics

- **Network training: 3 steps**
  - Forward propagation: we calculate the output values of neurons, from input layer to output layer:

$$y_i^l = f\left(\sum_{k \in [0,n]} w_{ik}^l . x_k^l + b_i^l \right)$$

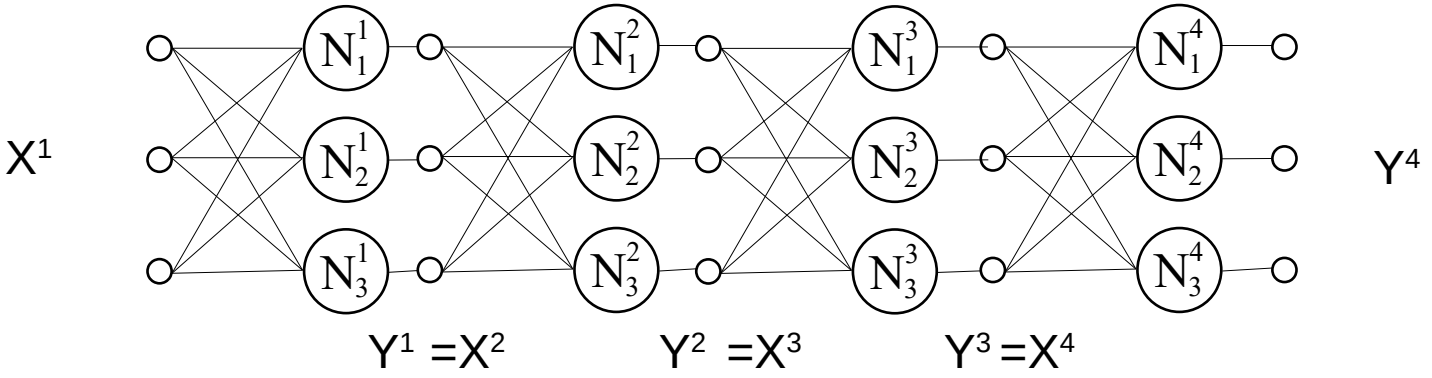
  - Backward propagation: we calculate the delta value of the output layer, then propagate deltas until the input layer
    - Last layer $\delta_i^L = f'_{(s_i^L)} . \left(r - y_i^L\right)$
    - Hidden layer $\delta_i^l = f'_{s_i^l} . \sum_{k \in [1,n]} \delta_k^{l+1} . w_{ki}^{l+1}$
  - Update of weights using deltas previously calculated

$$w_{ij}^l \Leftarrow w_{ij}^l + \alpha . x_j^l . \delta_i^l$$

$$b_i^l \Leftarrow b_i^l + \alpha . \delta_i^l$$

Artificial Intelligence for Robotics

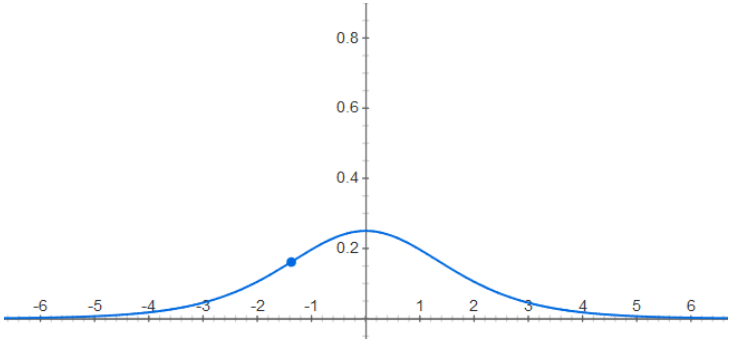
- **In practice:**
  - We can notice that if weights are initialized at 0,
    - Vectors Y^2 à Y^n will remain at 0
    - No possible learning
  - Weights of the network must be initialized with random values
    - Each learning will lead to a different result
    - With a sigmoïd activation function, we need to avoid great values, or the gradient may be too low



Artificial Intelligence for Robotics

- **In practice:**
  - With a sigmoïd activation function:
  - Derivative of sigmoïd function:

$$\sigma'(x) \quad = \quad \sigma(x) \ . \ (1-\sigma(x))$$



  - We already calculated σ during the *forward propagation*: they are the neuron's *y*
    - We thus obtain the following equation to calculate the deltas:

$$\delta_i^L \quad = \quad y_i^L \ . \ (1-y_i^L) \ . \ (r-y_i^L)$$

$$\delta_i^l \quad = \quad y_i^l \ . \ (1-y_i^l) \ . \ \sum_{k\in[1,n]} \delta_k^{l+1} \ . \ w_{ki}^{l+1}$$